



# Quick Login Reference - Trading Academy

---



## LOGIN CREDENTIALS

---

### ADMIN Account

Email: admin@tradingacademy.com  
Password: Admin@123456  
URL: <https://d4896f84e.preview.abacusai.app/login>

### REGULAR USER Account

Email: usuario@tradingacademy.com  
Password: Usuario@123456  
URL: <https://d4896f84e.preview.abacusai.app/login>

---



## QUICK NAVIGATION

---

### Admin Panel

- **Main Dashboard:** /dashboard/admin
- **Users & Roles:** /dashboard/admin/users
- **Leads & Contacts:** /dashboard/admin/leads
- **Courses:** /dashboard/admin/courses
- **AI Bots:** /dashboard/admin/bots
- **Security:** /dashboard/admin/security

### Client Area

- **Dashboard:** /dashboard
  - **My Courses:** /dashboard/mis-cursos
  - **Course Content:** /dashboard/curso/[courseId]
- 



## DATABASE INFO

---

**Database:** PostgreSQL

**Host:** localhost

**Port:** 5432

**Database Name:** postgres

**User:** postgres

**Password:** Cjjunior1

**Connection String:**

```
postgresql://postgres:Cjjunior1@localhost:5432/postgres
```

## Verify Users in DB

```
psql "postgresql://postgres:Cjjunior1@localhost:5432/postgres" \
-c "SELECT email, role, emailVerified FROM \"User\";"
```



## CREATE NEW USER (via Node.js)

```
cd /home/ubuntu/app && node -e "
const { PrismaClient } = require('@prisma/client');
const bcrypt = require('bcryptjs');
const prisma = new PrismaClient();

async function createUser(email, firstName, lastName, whatsappNumber, role = 'user') {
  const hashedPassword = await bcrypt.hash('Password@123456', 12);

  const user = await prisma.user.create({
    data: {
      email: email.toLowerCase(),
      password: hashedPassword,
      firstName,
      lastName,
      whatsappNumber,
      role,
      emailVerified: new Date()
    }
  });
};

console.log('✅ User created:', email, 'Role:', role);
await prisma.$disconnect();
}

createUser('newuser@example.com', 'John', 'Doe', '+5219876543210', 'user');
"
```



## SECURITY NOTES

- ✅ **Passwords are hashed** using bcryptjs (12 rounds)
- ✅ **Email verification required** before login
- ✅ **JWT tokens** stored in httpOnly cookies
- ✅ **Role-based access control** (admin vs user)
- ✅ **Admin routes protected** with `requireAdminSession()`



## KEY FILES

| File                                             | Purpose                     |
|--------------------------------------------------|-----------------------------|
| <code>lib/auth-options.ts</code>                 | NextAuth configuration      |
| <code>lib/admin.ts</code>                        | Admin protection middleware |
| <code>app/login/page.tsx</code>                  | Login form UI               |
| <code>app/registro/page.tsx</code>               | Registration form           |
| <code>prisma/schema.prisma</code>                | Database schema             |
| <code>app/api/auth/[...nextauth]/route.ts</code> | Auth API                    |



## TROUBLESHOOTING

### Login shows “invalid credentials”

- Check email spelling (case-sensitive)
- Verify password is exactly as provided
- Ensure emailVerified is set in database

### Can't access admin panel

- Verify user has `role = 'admin'`
- Check that session is active
- Clear browser cookies and retry

### Database connection issues

- Verify PostgreSQL is running
- Check DATABASE\_URL in .env
- Verify credentials in postgres connection



## APPLICATION URL

<https://d4896f84e.preview.abacusai.app>

**Status:**  Running

**Auth Method:** NextAuth.js + Credentials Provider

**Database:** PostgreSQL + Prisma ORM

**Last Updated:** May 2, 2026

**System:** Trading Academy (Next.js)